

FIELD MANUAL

stsmell Internals

*A function-level walk through a code smell detector for
Pharo/Smalltalk – every stage, every metric, every edge
case.*

subject stsmell 0.1.0
scope 2,416 lines across 23 modules
reads Tonel source, no Pharo image
format sized for a phone screen

CONTENTS

What is in here

- 01 The shape of the thing**
Layering law and module map

- 02 Entry and dispatch**
cli.py, four commands, exit codes

- 03 Configuration**
Discovery, parsing, three-level merge

- 04 Finding files**
discovery.py

- 05 Parsing**
tree-sitter, caching, the offset trick

- 06 Extraction**
CST to model, STON header, orphans

- 07 The model**
Dataclasses and one lifetime hazard

- 08 Selectors**
Why there is nothing to count

- 09 Method metrics**
One walk, nineteen counters

- 10 Class metrics**
Aggregation and cohesion

- 11 The hierarchy**
Rebuilding inheritance from names

- 12 The rule engine**
Registry, context, findings

- 13 Rule catalogue**
All sixteen, with conditions

- 14 Filter and report**
Baseline, floor, three renderers

- 15 One finding, end to end**
A single trace through every stage

- 16 Invariants and edges**
What holds, what silently does not

- 17 Extending it**
Adding a rule in four steps

SECTION 01

The shape of the thing

A strict one-way pipeline, with a hard rule about who may touch the parse tree.

Eight stages run in a fixed order. Data moves forward only; no stage reaches back to the one before it.

1 `cli.main`

Parse argv, dispatch to a command function.



2 `config.load_config`

Find and read stsmell.toml, merge defaults.



3 `discovery.discover`

Walk the tree for *.class.st and friends.



4 `parsing.parse_file`

tree-sitter produces one CST per file.



5 `extract.extract_file`

CST to ClassDef / MethodDef dataclasses.



6 `metrics.compute_metrics`

Hierarchy, then per-method, then per-class.



7 `rules.run_rules`

Every enabled rule against every entity.



8

`report.render_*`

Text, JSON or SARIF; exit 0, 1 or 2.

The layering law

One constraint shapes the whole design, and it is stated in the docstring of `rules/base.py`:

THE RULE

Rules read *computed metrics and the model only* – *never the CST*.

Contact with tree-sitter is confined to two modules, `parsing.py` and `extract.py`, plus the metric walkers that consume `body_node`. Above that line everything is plain numbers, strings and dataclasses.

Three things fall out of this:

- A rule is testable from a hand-built `ClassMetrics` with no file, no parser and no grammar.
- Swapping the grammar is a change to two files.
- A rule cannot accidentally depend on syntax, which would make its threshold meaningless across formatting styles.

Module map

MODULE	LINES	ROLE
cli.py	204	argparse, four commands
analysis.py	60	Pipeline orchestration
config.py	97	TOML, thresholds, severity order
discovery.py	44	File walk and excludes
parsing.py	112	tree-sitter, CST helpers
extract.py	206	CST → model
model.py	94	Dataclasses
selectors.py	95	Curated selector sets
metrics/method.py	324	Per-method counters
metrics/klass.py	95	Aggregation, TCC
metrics/hierarchy.py	80	Superclass graph
metrics/suite.py	44	Drives all metrics
rules/base.py	129	Rule, Finding, registry
rules/method_rules.py	258	9 method rules
rules/class_rules.py	216	5 class rules
rules/smalltalk_rules.py	129	3 idiom rules
report/text.py	92	Terminal, ANSI colour
report/sarif.py	79	SARIF 2.1.0
report/json_report.py	24	JSON

The orchestrator

`analysis.analyze` is the whole pipeline in nine lines, and is the function to call when embedding the tool:

```
def analyze(root, config=None):
    config = (config or
    Config()).with_defaults(registry())
    project, parse_errors, read_errors =
    build_project(root, config)
    suite = compute_metrics(project)
    findings = run_rules(suite, config)
    return AnalysisResult(...)
```

`build_project` wraps each file in a try/except for `OSError` and `UnicodeDecodeError`. A file that cannot be read is recorded in `read_errors` and skipped; the run continues.

SECTION 02

Entry and dispatch

Four subcommands over one analysis, distinguished by what they do with the result.

`build_parser` constructs an `argparse` tree with `required=True` on the subparsers, so bare `stsmell` is an error. A closure named `add_common` attaches the three shared arguments – `path`, `--config`, `--exclude` – to every subcommand that analyses source.

Each subparser sets `func` via `set_defaults`, and `main` simply calls `args.func(args)`. Adding a command means adding a parser and a function; nothing dispatches on strings.

The four commands

analyze

Report smells. Applies the baseline, then the severity floor, then renders in text, JSON or SARIF. The only command that can exit non-zero on findings.

metrics

Dump raw numbers as CSV or JSON, at method or class scope. Runs the rules too – it shares `analyze()` – but ignores the findings. This is the recalibration tool.

rules

List every registered rule with its resolved thresholds. Takes no path: it loads config and the registry only.

baseline

Freeze the current findings to JSON so a later run reports only what is new.

Exit codes

CODE	MEANING
0	Clean, or a non-analyze command succeeded
1	At least one finding survived filtering
2	ConfigError or FileNotFoundError

Note the asymmetry: **unreadable files do not change the exit code**. They are printed to `stderr` and counted, but `cmd_analyze` returns `EXIT_FINDINGS` purely on whether the findings list is non-empty. A CI job that must fail on read errors has to check `stderr` itself.

DELIBERATE

`BrokenPipeError` is caught in `main` and returns 0, so `stsmell analyze src/ | head` does not produce a traceback.

Order of filtering

In `cmd_analyze` the baseline is subtracted *before* the severity floor is applied. The two are independent, but the ordering means a baselined critical finding is gone by the time the floor is consulted – it can never resurface by lowering `min_severity`.

Configuration

Three sources, merged in a fixed precedence, resolved lazily at the point of use.

Finding the file

`find_config` starts at the analysed path – its parent if it is a file – and walks upward through every ancestor directory, checking for `stsmell.toml` then `.stsmell.toml` at each level. The first hit wins. An explicit `--config` bypasses the search entirely.

This means a monorepo can hold one config at the root, and analysing any subdirectory picks it up.

Reading it

`load_config` imports `tomllib` on 3.11+ and falls back to the third-party `tomli` on 3.10. If neither is present the error is raised at read time, not import time, so a run with no config file works on 3.10 without `tomli` installed.

Validation is deliberately shallow. Three things are checked:

- `min_severity` must be one of the four known levels.
- `[rules]` must be a table.
- Each `[rules.x]` must be a table.

Threshold *names* are not validated. A typo such as `max_line = 30` is silently stored and never read; the rule keeps its default.

This is a real trap, and the reason `stsmell rules` prints resolved thresholds.

Three-level resolution

`with_defaults` runs once per analysis, before any rule executes. For each registered rule it builds a merged dict:

```
merged = dict(rule.defaults)           # 1. class defaults
merged.setdefault("enabled", True)
merged.update(self.rules.get(rule.id, {})) # 2. TOML
wins
```

At check time, `Rule.opt` reads through that merged view:

```
def opt(self, ctx, name):
    return ctx.config.threshold(
        self.id, name, self.defaults.get(name))
```

The third fallback — `self.defaults.get(name)` — matters when a config file was loaded but never passed through `with_defaults`, as in a unit test. It is why every rule body must call `self.opt(ctx, ...)` rather than reading `self.defaults` directly.

Command-line flags are layered on last, inside `cli._load`: `--exclude` extends the list, `--min-severity` overwrites.

SECTION 04

Finding files

Forty-four lines, three suffixes, one exclusion that matters.

Tonel keeps one type per file, so file discovery *is* class discovery.

`is_tonel_file` accepts three suffixes:

```
TONEL_SUFFIXES = (".class.st", ".trait.st",  
                  ".extension.st")
```

and rejects one name outright:

```
if name == "package.st":  
    return False # marker, no class or method
```

`discover` handles a file path and a directory path through the same entry point: given a file it returns `[root]` or `[]`, given a directory it runs `rglob("*.*st")` and filters. The result is sorted, which is what makes output stable across runs and filesystems.

Excludes

Four patterns are always applied – `.git`, `node_modules`, `.venv`, `pharo-local` – with user patterns appended. Matching is `fnmatch` against the *full resolved path string*, so a pattern needs leading and trailing wildcards to match a path segment:

`*/Tests/*`, not `Tests`.

FORMAT TRAP

A filetree repository – the older `.package/` layout used by Seaside and zinc – contains no `.class.st` files at all. `discover` returns an empty list, every later stage runs cleanly over nothing, and the tool reports zero findings and exits 0. Check for a `.properties` file holding `{ #format : #tonel }` before trusting a clean run.

Parsing

A real CST, built once per file, with one rewrite that preserves every byte offset.

Caching

Both grammar and parser are memoised with

`functools.lru_cache(maxsize=1)`, so the shared library loads once per process and a single `Parser` instance serves every file. The comment on `get_language` records a specific gotcha: the package's `language()` already returns a built `tree_sitter.Language`, and wrapping it again raises `TypeError`.

The offset-preserving rewrite

Traits write class-side methods as `TFoo classSide >> bar`, but the grammar only knows the class form `Foo class >> bar`. Such files would fail to parse. `normalize_source` rewrites the keyword before parsing:

```
_CLASS_SIDE_RE = re.compile(
    rb"^(^|\n)([A-Za-z_]\w*)([ \t]+)classSide([ \t]*>>)"
    _REPLACEMENT = rb"\1\2\3class    \4"
```

The substitution is the point. `classSide` is nine characters; `class` plus four spaces is also nine. Every byte after the rewrite sits at exactly the offset it had in the original file, so **every reported line and column is still true of the file on disk**. A shorter replacement would silently shift columns on the rewritten line.

Errors are recorded, not raised

tree-sitter recovers from syntax errors rather than aborting, producing a partial tree with `ERROR` nodes in it. `has_error` reads `tree.root_node.has_error`, and `build_project` stores the result on `ParsedFile.parse_error` and appends the path to a list.

The file stays in the project. Partial results from a partially parsed file are still useful, and the count surfaces in the report footer rather than failing the run.

Helper inventory

`node_text(node, source)`

Slice by byte offsets, decode UTF-8 with `errors="replace"`. Returns `""` for `None`.

`strip_string_literal`

`'it's'` → `it's`.

`strip_symbol`

`#name` → `name`; handles the quoted form `#'odd name'`.

`strip_class_comment`

Unwraps double quotes, halves doubled `""`.

`walk / walk_named`

Iterative depth-first generators over a node and its descendants. Iterative, not recursive – no stack limit on a deep tree.

`children_of_type / first_child_of_type`

Direct-child lookup by node type.

Extraction

The bridge: tree-sitter nodes in, plain dataclasses out.

Reading the STON header

A Tonel definition header is STON, a Smalltalk-flavoured JSON. Three small functions read it:

`_ston_pairs(map_node)`

Maps each key, with `#` stripped, to its unparsed `ston_value` node.

`_ston_scalar(value_node)`

Dispatches on the first character: `'` means string, `#` means symbol, anything else is returned raw (numbers).

`_ston_list(value_node)`

Reads a `ston_list` of scalars. If there is no list node it degrades to a single scalar wrapped in a list, tolerating `#instVars : 'x'`.

`_build_class` then pulls the known keys. One line encodes a format migration:

```
# Tonel 3.0 replaced #category with
# #package + #tag for class definitions.
package=scalar("package") or scalar("category"),
category=scalar("category") or scalar("tag"),
```

Both spellings resolve to the same fields, so a repository written before or after the change measures identically.

Reconstructing selectors

A keyword method declares one `selector` field per keyword part. `_selector_of` concatenates them:

```
parts = [node_text(n, source).strip()
          for n in
          ref.children_by_field_name("selector")]
return "".join(parts), params
```

So `deposit: amount into: account` yields the selector `deposit:into:` and the params `['amount', 'account']`. That concatenated form is the identity used everywhere downstream – in findings, in `refused-bequest` matching, in the CSV.

Method metadata

`_build_method` reads the `{ #category : '...' }` pragma above the method, then scans the direct children of `method_body` for two node types: `temporaries` (contributing declared names) and `pragma` (contributing `<primitive:>`-style annotations).

CONSEQUENCE

Only *direct* children are scanned, so a temporary declared inside a block does not appear in `method.temporaries` and does not count toward TEMPS.

The assembly loop

`extract_file` iterates the top-level children of the root node and handles four cases:

1. `class_comment` – held in `pending_comment` and attached to the next definition seen.
2. `definition` – unwrapped to find the inner class, trait or extension node.

3. A bare definition node – tolerated in case the grammar does not wrap it.
4. `method_definition` – built and filed against its owning class.

Orphan methods get a home

`_target_class` matches a method to its class by name. Two fallbacks keep methods from being dropped:

- If exactly one definition exists in the file, an unmatched method is filed there anyway – Tonel guarantees one type per file, so a renamed or mismatched reference still lands sensibly.
- If there is no definition at all, a synthetic `ClassDef` of kind `extension` is created to hold the method, so it is still analysed.

SECTION 07

The model

Four dataclasses, and one reference that must not be dropped.

MethodDef

class_name, selector, params, temporaries, pragmas, category, class_side, path, start_line, end_line, source, and `body_node`.

ClassDef

name, superclass, inst_vars, class_inst_vars, class_vars, traits, package, category, kind, comment, methods. Properties: `is_extension`, `instance_methods`, `class_methods`.

ParsedFile

path, source, tree, classes, parse_error.

Project

root and files, with `classes`, `methods` and `class_names()` flattening across files.

The lifetime hazard

`ParsedFile` carries the whole `tree`, and its docstring says why:

DO NOT OPTIMISE THIS AWAY

Holds `tree` so that tree-sitter Nodes referenced by `MethodDef` stay alive; **dropping the Tree invalidates every Node derived from it.**

Nodes are views into memory owned by the C-level tree. If the last Python reference to the `Tree` is released, every `body_node` in every `MethodDef` becomes a dangling handle. The metric pass runs long after extraction, so the tree has to survive until then –

which also means peak memory is proportional to the whole corpus, not one file.

Two distinctions worth noting

`klass.methods` holds *both* sides. `instance_methods` and `class_methods` partition it on `m.class_side`. So NOM counts both sides while NOCM counts only the class side – the two are not disjoint.

`is_extension` is a kind check, not a heuristic. It gates three class rules that would otherwise measure a fragment as though it were a class.

Selectors

The Smalltalk-specific heart of the tool, and the reason a Java linter cannot be ported directly.

Smalltalk has no control-flow syntax. No `if`, no `while`, no `for`. Every branch and loop is an ordinary message send taking block arguments:

```
x > 0
    ifTrue: [ self grow ]
    ifFalse: [ self shrink ].
```

There is no keyword to count. Cyclomatic complexity has to be computed by recognising *selectors* on message-send nodes, which is what `selectors.py` curates. Four sets union into **BRANCHING**:

SET	SIZE	EXAMPLES
CONDITIONAL	20	<code>ifTrue:</code> , <code>ifNil:</code> , <code>detect:ifNone:</code> , <code>at:ifAbsent:</code>
BOOLEAN	2	<code>and:</code> , <code>or:</code>
LOOPING	23	<code>do:</code> , <code>collect:</code> , <code>to:do:</code> , <code>inject:into:</code>
EXCEPTIONAL	5	<code>on:do:</code> , <code>ensure:</code> , <code>ifCurtailed:</code>

Two smaller sets steer other metrics:

ABSTRACT_MARKERS

`subclassResponsibility`, `shouldBeImplemented`, `notYetImplemented`. A body consisting solely of one of these marks an abstract or refused method.

SELF_RECEIVERS

`self`, `super`, `thisContext`. Receivers that are not foreign objects, so they are excluded from feature envy and Law of Demeter.

Node-type sets complete the picture: `MESSAGE_NODES` is the three ordinary send types, `CASCADED_MESSAGE_NODES` the three cascade forms whose receiver is implicit, and `ALL_MESSAGE_NODES` their union.

IMPLICATION

Coverage is a curation problem, not a parsing problem. A project with its own iteration selector – say `eachPairDo:` – is undercounted for complexity until that selector is added to the set. The lists are the tool's calibration surface.

Method metrics

One pass over the body populates nineteen fields; two small passes clean up what the first cannot see.

`compute_method_metrics(method, klass, source)` returns a `MethodMetrics`. It begins with the two counts that need no tree – `params` and `temps`, both lengths of lists already on the model – and returns early if `body_node` is `None`.

Setting up

Two name sets are built before the walk, and the difference between them is deliberate:

```
ivars = inst_vars | class_inst_vars | class_vars
envy_names = set(method.params)
```

The comment explains the asymmetry: feature envy is about reaching into *other* objects, and a temporary the method constructed itself – `parts := OrderedCollection new` – is not a foreign object. So temporaries are excluded from envy candidates while instance state and parameters are not.

A third set solves a subtler problem:

```
declared_ids = {id(ident)
  for node in walk(body) if node.type == "temporaries"
  for ident in node.children
  if ident.type == "identifier"}
```

Identifiers inside a `| a b |` declaration are declarations, not accesses, and would otherwise inflate NOAV. Note the key is

`id(node)` – object identity, not name – so a variable that is both declared and used still counts its uses.

The main walk

A single `walk(body)` dispatches on node type.

MESSAGE NODES

Increment MSG. Reconstruct the selector; if it is in `BRANCHING`, increment CYCLO. Then, for non-cascaded sends only:

- Compute chain length and update MAXCHAIN.
- Find the chain base; if it is not a self-receiver, update FOREIGN_MAXCHAIN.
- Look at the immediate receiver: a self-receiver increments SELF_SENDS; a plain identifier that names an ivar or a parameter increments that name's entry in the `foreign_sends` Counter.

TWO EXCLUSIONS IN ONE CONDITION

The receiver branch reads `elif receiver.type == "identifier" and kind != "binary_message"`. Binary sends are arithmetic and comparison – `amount > 0` – not delegation, so they never indicate envy. And only lowercase identifiers reach the counter, because a send to a capitalised global is construction, not envy.

CASCADE NODES

MAX_CASCADE takes the maximum of `1 +` the count of cascaded children – the one plus counts the head message before the first semicolon.

THISCONTEXT NODES

Set the boolean. No threshold; presence is the finding.

IDENTIFIER NODES

Skipped if in `declared_ids`. Otherwise the first character decides: uppercase goes to `referenced_classes`, which later becomes CBO; lowercase goes to the accessed set, which becomes NOAV, and additionally to `accessed_ivars` if it names instance state – the input to class cohesion.

The tree walkers

`_unwrap(node)`

Descends through `expression`, `primary` and `statement` wrapper nodes while each has exactly one named child. Without it every structural test would have to know the grammar's wrapper layers.

`message_selector(node)`

Three cases. Unary and binary read a single selector child; keyword concatenates every `keyword` child, reproducing `at:put:`.

`_chain_length(node)`

Starts at 1 and follows the `receiver` field while it is itself a message node.

`_chain_base(node)`

Same walk, but returns the final non-message receiver – the root object the chain started from.

`_block_depth(body)`

Recursive descent tracking depth, incrementing on each `block` node, returning the maximum.

`_code_lines(body, source)`

Collects line numbers from *leaf* nodes only, skipping comments, into a set. Counting distinct lines rather than nodes means

MLOC is a physical line count that ignores blanks and comment-only lines.

Shape detectors

`_detect_accessor` recognises the two canonical shapes and nothing else. It requires exactly one statement and no temporaries, then matches either:

- `^ ivar` – a return, no params, of an identifier naming an instance variable; or
- `ivar := arg` – an assignment, exactly one param, whose target is an ivar and whose value is that same parameter by name.

The value check is strict: `ivar := arg asString` is not an accessor, because the assigned value is a message send rather than the bare parameter.

`_detect_abstract` requires one statement which, after optionally unwrapping a return, is a unary message whose receiver is `self` and whose selector is in `ABSTRACT_MARKERS`. Both `self subclassResponsibility` and `^ self subclassResponsibility` match.

The second pass

After the main walk, a separate loop looks for a send to `super` with the same selector as the enclosing method, setting `calls_super_same_selector`. It breaks on the first hit. This is a distinct pass because it needs the method's own selector compared against each send – a relation the type-dispatching main loop is not shaped to express.

The nineteen fields

FIELD	MEANING
mloc	Code lines
cyclo	Complexity, starts at 1
msg	Total sends
params	Selector arity
temps	Method-level temporaries
nblk	Max block depth
maxchain	Longest chain, any root
foreign_maxchain	Longest non-self chain
noav	Distinct names accessed
max_cascade	Longest cascade
self_sends	Sends to self/super
foreign_sends	Counter by receiver name
accessed_ivars	Set, feeds TCC
referenced_classes	Set, feeds CBO
uses_this_context	Boolean
is_accessor	Boolean
is_abstract	Boolean
calls_super_same_selector	Boolean

The derived property `top_foreign` returns the most-messaged receiver and its count, or `(None, 0)`.

Class metrics

Mostly aggregation. One measure does real work.

`compute_class_metrics` takes the class, its method metrics, the hierarchy and the set of project class names. Seven fields are direct aggregation:

```
nom  = len(klass.methods)
nocm = len(klass.class_methods)
niv  = len(klass.inst_vars)
wmc  = sum(m.cyclo for m in method_metrics)
cloc = sum(m.mloc for m in method_metrics)
dit  = hierarchy.depth.get(klass.name, 0)
noc  = hierarchy.num_children(klass.name)
```

WMC is worth restating: it is the *sum of complexities*, not a method count. Forty trivial accessors give $WMC \approx 40$; ten branch-heavy methods can exceed it.

Coupling

```
referenced = union of m.referenced_classes
coupled = (referenced & project_classes) -
    {klass.name}
cbo = len(coupled)
```

The intersection is the whole design decision. Every capitalised identifier was collected during the method walk, including `OrderedCollection` and `Transcript`; intersecting with classes actually present in the run discards kernel references. Coupling is therefore measured relative to *the analysed source*, and narrowing the scanned path lowers CBO for the same code.

Tight class cohesion

TCC asks: of all pairs of methods, what fraction touch a common instance variable?

```
eligible = [m for m in metrics
            if not m.is_accessor
            and not m.is_abstract
            and not m.method.class_side]
if len(eligible) < 2: return None
pairs = combinations(eligible, 2)
connected = sum(1 for a, b in pairs
               if a.accessed_ivars & b.accessed_ivars)
return connected / len(pairs)
```

1.0 is perfect cohesion; near zero means the methods are strangers sharing a file. Three exclusions and one refusal:

- **Accessors** touch exactly one ivar and would connect to everything that uses it.
- **Abstract methods** touch none and would connect to nothing.
- **Class-side methods** do not access instance state at all.
- Under two eligible methods, TCC is `None` – *undefined, not zero*. Every consumer must guard, and `god-class` does.

Cost is quadratic in eligible methods. For a 200-method class that is 19,900 set intersections – acceptable, but it is the hot spot.

Finally, `acc_ratio` is the share of methods flagged `is_accessor`, guarded against division by zero.

SECTION 11

The hierarchy

Tonel records no inheritance graph. It is rebuilt at runtime by matching strings.

Each file names its superclass as a string in its own header. Nothing indexes them, so `Hierarchy.build` reconstructs the graph in three passes over `project.classes`:

1. Index every non-extension class by name into `by_name`, using `setdefault` so the first definition wins on a name collision.
2. For each class, append its name to `children[superclass]`; if the superclass is absent from `by_name`, record it in `unresolved`.
3. Compute `depth[name]` for every class.

What a depth of 1 means

`_compute_depth` counts links walked *until the chain leaves the project*:

```
depth = 0
seen = {name}
while current is not None:
    klass = by_name.get(current)
    if klass is None or not klass.superclass:
        break
    depth += 1
    parent = klass.superclass
    if parent in seen: break # cyclic
    seen.add(parent)
    current = parent
```

A class whose superclass is `Object` has depth 1, because `Object` lives in the kernel and is not in `by_name`. The docstring is explicit that chains leaving the project stop rather than guessing at the depth of the external hierarchy.

So DIT under-reports, systematically, by however deep the real kernel ancestry runs. Analysing one package of a large project gives smaller numbers than the image would report. The `max_depth` default of 8 is calibrated against that convention, not against absolute image depth.

The `seen` set is a defensive guard against a malformed cyclic hierarchy – a real possibility when the graph is assembled from unvalidated strings rather than a checked structure.

SECTION 12

The rule engine

A registry of stateless singletons, three optional hooks, one triple loop.

Registration

```
def register(rule_cls):  
    _REGISTRY.append(rule_cls())  
    return rule_cls
```

The decorator **instantiates at import time** and stores the instance. One object per rule serves the whole run, so rules must be stateless – nothing may accumulate on `self` between calls. Anything cross-class goes through `check_project` and the context instead.

Import order determines registry order, which determines the order of same-severity findings after sorting.

`rules/__init__.py` imports the three rule modules for their side effect.

The three hooks

`Rule` declares all three returning `()`, so a subclass overrides only the level it needs:

`check_method(mm, cm, ctx)`

Per method. Receives the class metrics too, so a method rule can consult its owner.

`check_class(cm, ctx)`

Per class.

`check_project(ctx)`

Once per run. Only `extension-sprawl` uses it.

All three are declared `Iterable[Finding]`, which is why bodies use `yield`: a rule that finds nothing yields nothing, and no empty-list bookkeeping is needed.

The three arguments

`self`

The shared singleton. Carries `id`, `severity`, `defaults`, plus `opt()` and the two finding builders.

`cm / mm`

The subject. Computed numbers, the underlying model object, and for a class the list of its method metrics.

`ctx`

Two fields. `ctx.config` for thresholds, reached through `self.opt`. `ctx.suite` for the whole-project view – the hierarchy and every class – which is how `refused-bequest` reaches a parent class living in another file.

The driver

```
ctx = RuleContext(suite=suite, config=config)
active = [r for r in registry()
          if config.enabled(r.id)]

for rule in active:
    findings += rule.check_project(ctx)

for cm in suite.classes:
    for rule in active:
        findings += rule.check_class(cm, ctx)
    for mm in cm.method_metrics:
        for rule in active:
            findings += rule.check_method(mm, cm, ctx)
```

Disabled rules are filtered once, up front, so they are never called. Project rules run first, before any per-entity work.

Findings

A `Finding` carries rule id, severity, message, entity, file, line and a free-form metrics dict. Two helpers build one, filling identity and location from the subject; keyword arguments become the metrics dict that reaches JSON and SARIF.

Two derived pieces matter:

`severity_rank`

Numeric, via `SEVERITY_ORDER`, for the floor comparison and the sort.

`key()`

`(rule, entity)` – deliberately **line-number independent**, so a baseline survives edits that move code around.

The final sort is `(-severity_rank, file, line, rule)`: worst first, then grouped by file in line order.

SECTION 13

Rule catalogue

Sixteen rules. Each card gives the smell and the exact firing condition.

Method rules — 9

long-method **MAJOR**

Body too long to read in one sitting.

```
MLOC > max_lines · default 25
```

complex-method **MAJOR**

Too many independent paths. Counted from branching selectors, not keywords.

```
CYCLO > max_cyclo · default 8
```

long-parameter-list **MINOR**

Keyword message takes more arguments than a caller can track.

```
PARAMS > max_params · default 4
```

too-many-temporaries **MINOR**

Enough local state to suggest several jobs in one method.

```
TEMPS > max_temps · default 6  
block-local temps not counted
```

deeply-nested-blocks MAJOR

Control flow buried several levels deep; each block is a conditional or loop body.

```
NBLK > max_depth · default 3
```

message-chain MINOR

Law of Demeter. Navigates a long path through other objects.

```
FOREIGN_MAXCHAIN > max_chain · default 4  
chains rooted at self/super excluded
```

feature-envy MAJOR

More interested in another object than its own.

```
top foreign ≥ min_foreign_sends (5)  
AND top foreign > SELF_SENDS  
accessors exempt
```

uses-this-context MINOR

Reflective stack access. Legitimate in debuggers and exception machinery, a smell elsewhere.

```
boolean – no threshold
```

cascade-abuse INFO

One cascade configures an object at unreadable length.

```
MAX_CASCADE > max_cascade (6)  
disabled in the shipped config
```

Class rules — 5

god-class **CRITICAL**

Does too much, holds too much, hangs together poorly. Composite strategy after Lanza & Marinescu.

```
WMC ≥ 47 AND TCC < 0.33 AND NIV ≥ 5  
skipped when TCC is None or extension
```

large-class **MAJOR**

More methods than one abstraction warrants. Kept separate from god-class: bulk alone is not a fault.

```
NOM > max_methods · default 40
```

data-class **MINOR**

A record with getters; the behaviour that should use this data lives elsewhere.

```
NOM ≥ 5 AND NIV ≥ 2  
AND ACC_RATIO ≥ 0.8
```

refused-bequest **MAJOR**

A subclass rejects the interface it inherits.

```
IS_ABSTRACT method whose selector is in the parent's  
instance-side selectors  
silent if superclass outside source
```

deep-inheritance **MINOR**

Behaviour spread over a long ancestor chain.

```
DIT > max_depth (8), within source only
```

high-coupling **MINOR**

Depends on too much of the system to change safely.

```
CBO > max_cbo (15), project classes only
```

Smalltalk rules — 3

missing-class-comment **MINOR**

Pharo treats the class comment as primary documentation, and the community enforces it.

```
comment absent or blank  
classes and traits only
```

god-class-side **MINOR**

The metaclass has become a utility bag of static-style helpers.

```
NOCM > max_class_methods (15)
```

extension-sprawl **MAJOR**

A package monkey-patches foreign classes at a scale that hides behaviour. The one project-scoped rule.

```
extended classes > 10  
OR extension methods > 30  
per package
```

The omission worth knowing

refused-bequest implements half the classic heuristic on purpose. The Java version also flags subclasses that never touch inherited instance variables. The code says why that half is absent:

MEASURED, NOT ASSUMED

Idiomatic Smalltalk reaches inherited state through accessors – `self origin` – rather than direct variable reference. On the Pharo corpus the ivar heuristic produced **3,707 findings against 16 real ones**.

So the rule fires only on the unambiguous signal: overriding an inherited method with `subclassResponsibility`.

Filter and report

Two subtractions, then one of three renderers.

Baseline subtraction

`_load_baseline` reads the JSON and builds a set of `(rule, entity)` tuples. Findings whose `key()` is in that set are dropped.

Because the key omits the line number, moving a method within its file does not resurrect a baselined finding. The trade-off is granularity: two findings of the same rule on the same entity collapse to one key, so if a method trips `long-method` twice – impossible today, but possible for a future rule that yields more than once – the baseline suppresses both.

Severity floor

`SEVERITY_ORDER` maps info 0, minor 1, major 2, critical 3. Findings below `min_severity` are dropped. Applied after the baseline.

Three renderers

`render_text`

Groups by file, prints line, padded severity, message, rule id, then the entity on a second line. Colour is enabled only when the stream is a TTY and `NO_COLOR` is unset. The footer counts findings by severity plus files, classes and methods, and appends warnings for parse and read errors.

`render_json`

Root path, a summary block of six counts, the parse-error paths, and every finding as a dict.

`render_sarif`

SARIF 2.1.0 for GitHub code scanning.

Two details in the SARIF writer

SARIF has three levels where the tool has four severities, so they are folded: critical and major both become `error`, minor becomes `warning`, info becomes `note`. Critical and major are indistinguishable once uploaded.

The rules array is filtered to `if rule.id in used` – only rules that actually fired are declared, keeping the payload small.

And an em dash separates entity from message rather than a colon, with a comment explaining why: keyword selectors already end in `:`, so a colon separator would render as `classify:: ...`.

Paths are made relative to the project root, falling back to the absolute path when `relative_to` raises. Lines are clamped with `max(f.line, 1)`, since SARIF regions are 1-based and a synthesised holder class can carry line 0.

One finding, end to end

A single trip through all eight stages, using a fixture in the repository.

The source, `tests/fixtures/smelly/`
`FeatureEnvyExample.class.st`:

```
{ #category : 'printing' }
FeatureEnvyExample >> describeCustomer [
    | parts |
    parts := OrderedCollection new.
    parts add: customer firstName.
    parts add: customer lastName.
    parts add: customer emailAddress.
    parts add: customer phoneNumber.
    parts add: customer postalAddress.
    parts add: customer accountNumber.
    ^ parts
]
```

- 1. Discover.** Suffix `.class.st` matches; the path is sorted into the list.
- 2. Parse.** No `classSide`, so `normalize_source` changes nothing, tree-sitter returns a clean tree.
- 3. Extract.** The STON header yields name, superclass `Object`, one instance variable `customer`. The method is built with selector `describeCustomer`, no params, temporaries `['parts']`, and `body_node` retained.
- 4. Set up.** `ivars = {'customer'}`. `envy_names = set()` – no params. `declared_ids` holds the `parts` identifier inside the temporaries node.

5. **Walk.** Six `add:` keyword sends and six unary sends to `customer`. Each unary send has receiver type `identifier`, name `customer`, which is in `ivars` – so `foreign_sends['customer']` reaches 6. No receiver is a self-receiver, so `self_sends` stays 0. `OrderedCollection` is capitalised and lands in `referenced_classes`.
6. **Detect.** Two statements plus a return, and a temporary is declared, so `is_accessor` is False.
7. **Judge.** `FeatureEnvy.check_method` reads `top_foreign = ('customer', 6)`. Not an accessor; $6 \geq 5$; $6 > 0$. It yields.
8. **Report.** Severity major, rank 2, sorted above the minors, printed under its file with the entity beneath.

```
smelly/FeatureEnvyExample.class.st
    9      major  sends 6 messages to 'customer'
          but only 0 to self  [feature-envy]
    FeatureEnvyExample>>describeCustomer
```

Exit code 1.

Invariants and edges

What the code guarantees, and where it quietly gives up.

Invariants

- **Byte offsets are never shifted.** The only source rewrite is length-preserving, so every line and column is true of the file on disk.
- **Discovery output is sorted,** so findings are stable across runs and filesystems.
- **The tree outlives extraction.** `ParsedFile.tree` keeps every `body_node` valid.
- **Rules never see the CST** and are stateless singletons.
- **Baseline keys ignore line numbers,** so they survive code movement.

Silent degradations

SITUATION	BEHAVIOUR
Filetree repo, not Tonel	Zero files, zero findings, exit 0
Superclass outside source	DIT stops; refused-bequest silent
Under 2 eligible methods	TCC None; god-class skipped
Misspelled threshold key	Stored, never read, default used
Unreadable file	stderr note; exit code unchanged
Parse error	Partial tree kept; counted in footer
Unknown iteration selector	CYCLO under-counts
Narrower scan path	CBO and DIT both fail

Cost

Everything is linear in source size except TCC, which is quadratic in a class's eligible method count. Memory is proportional to the whole corpus, because every parse tree is retained until the run ends.

Extending it

Adding a rule touches one file, if the metric already exists.

1. Pick the level: method, class or project.
2. If the number you need is not already on `MethodMetrics` or `ClassMetrics`, add a field and populate it in the existing walk – do not start a second traversal.
3. Subclass `Rule` in the matching module, decorate with `@register`, set `id`, `name`, `severity`, `description`, `defaults`.
4. Implement one hook. Read thresholds with `self.opt(ctx, ...)`, never `self.defaults`. Yield through `self.method_finding` or `self.class_finding`.

```
@register
class ShoutingSelector(Rule):
    """Selector is written in caps."""
    id = "shouting-selector"
    name = "Shouting Selector"
    severity = "info"
    description = "Selector is all uppercase."
    defaults = {}

    def check_method(self, mm, cm, ctx):
        sel = mm.method.selector
        if sel and sel.isupper():
            yield self.method_finding(
                mm, f"selector {sel} is shouting")
```

The rule appears in `stsmell rules` immediately, is configurable under `[rules.shouting-selector]` with no extra wiring, and can be switched off with `enabled = false`.

BEFORE YOU COMMIT

The suite asserts every registered rule has a non-empty `description`. It is user-visible: it reaches `stsmell rules` and the SARIF `fullDescription`. Implementation notes belong in the class docstring instead.

— END —